

10 Software Engineering Misconceptions Every Developer Should Know👁👁

Understanding these will make you a better developer.

Misconception #1

GitHub = Git

Reality

Git and GitHub solve different problems.

Git

- Tracks changes in your code
- Works completely offline
- Lives on your computer

GitHub

- Hosts Git repositories
- Makes collaboration possible
- Adds pull requests, issues, reviews and CI integrations

Why it matters

If GitHub goes down, Git still works locally.

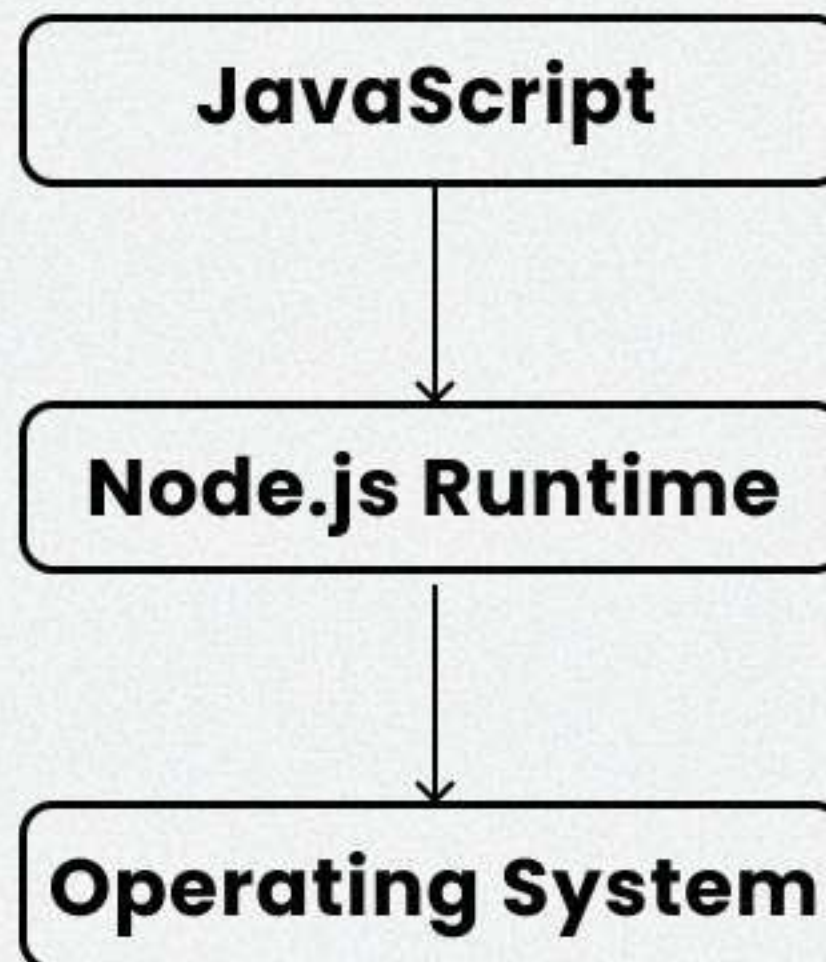
Understanding the difference helps you troubleshoot and collaborate more effectively.

Misconception #2

Node.js is a programming language

Reality

Node.js is a JavaScript runtime, not a language. JavaScript defines the syntax and language features. Node.js provides the environment to run JavaScript outside the browser, along with APIs for files, networking, processes and servers.



Why it matters

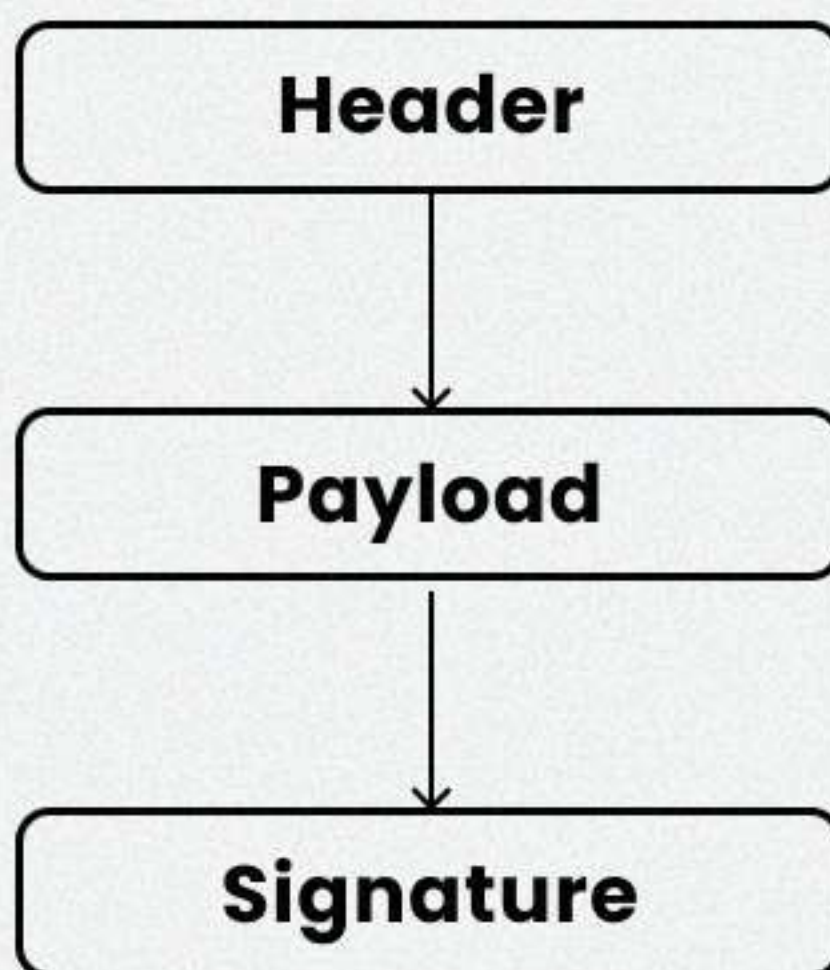
It explains why browser APIs and Node.js APIs aren't always the same.

Misconception #3

JWTs are encrypted

Reality

A standard JWT is signed, not encrypted.



Anyone who has the token can decode the payload. The signature only proves the token hasn't been modified. Never store passwords, secrets or sensitive personal information inside a JWT payload.

Why it matters

Many security mistakes happen because developers assume JWTs hide data.

Misconception #4

Docker is a Virtual Machine

Reality

Containers and virtual machines isolate applications differently.

Virtual Machine

Runs a complete guest operating system.

Container

Shares the host operating system's kernel while isolating the application and its dependencies. This is why containers are generally faster to start and consume fewer resources.

Why it matters

Knowing the difference helps you choose the right deployment strategy.

Misconception #5

HTTPS is a different protocol from HTTP

Reality

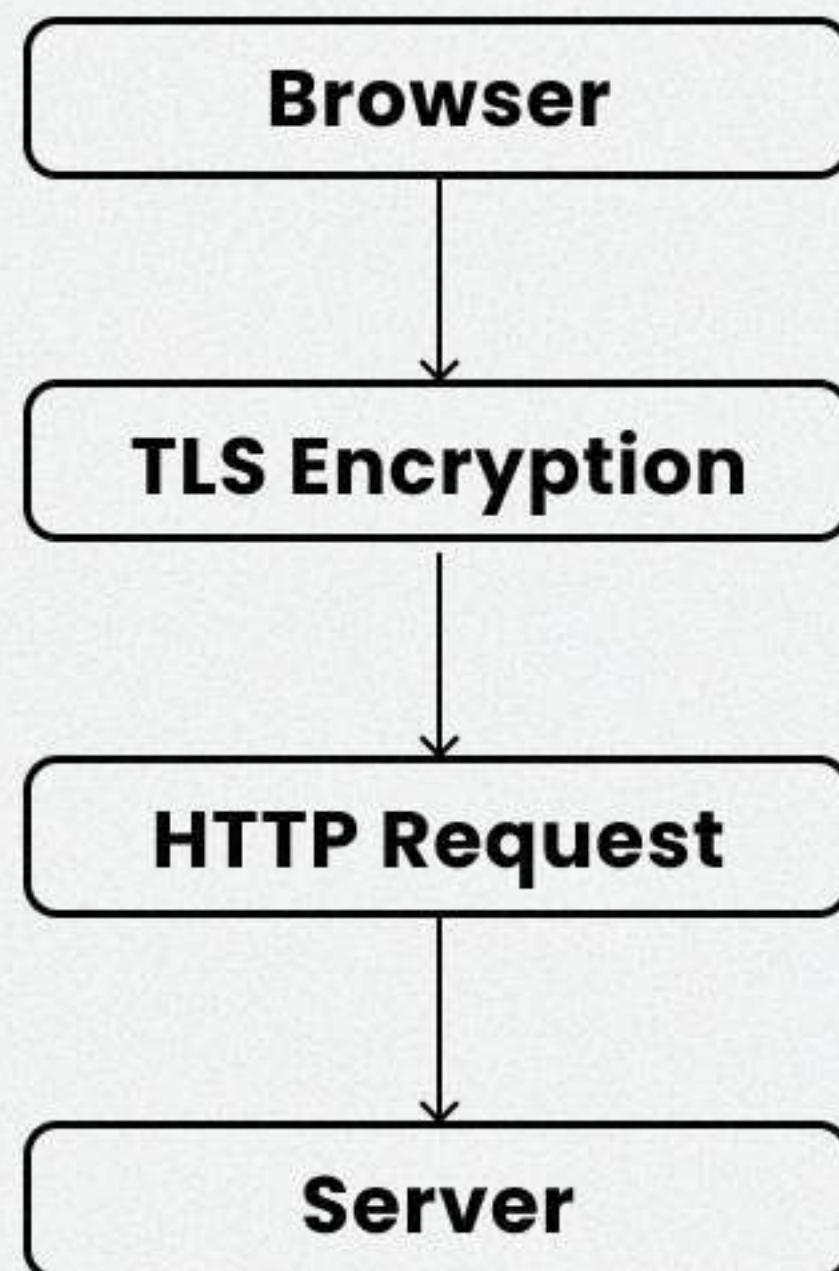
HTTPS is HTTP running over TLS.

HTTP still handles requests and responses.

TLS protects them while they're travelling across the network.

Why it matters

Understanding this makes HTTPS, certificates and secure communication much easier to learn.



Misconception #6

REST is an API

Reality

REST is an architectural style for designing APIs.

An API is simply a way for software systems to communicate.

REST is one way to design an API.

Other approaches include:

- GraphQL
- gRPC
- SOAP

Why it matters

This distinction helps you compare technologies correctly instead of treating REST as the only type of API.

Misconception #7

SQL is a database

Reality

SQL is a language.

Databases store data.

SQL lets you query and manage that data.

Examples of relational databases:

- PostgreSQL
- MySQL
- SQLite
- SQL Server

Why it matters

Learning SQL doesn't automatically mean learning databases—you also need to understand indexing, transactions, constraints and schema design.

Misconception #8

More RAM makes an application faster

Reality

Performance bottlenecks depend on the workload.

Sometimes the problem is:

- inefficient algorithms
- slow database queries
- missing indexes
- network latency
- blocking operations
- excessive disk I/O

Adding hardware can't fix inefficient software.

Why it matters

Optimise first. Scale second.

Misconception #9

AI understands your code

Reality

AI models predict likely outputs based on patterns in data.

They can generate impressive code—but they can also hallucinate APIs, misunderstand requirements or introduce subtle bugs.

Why it matters

Treat AI like a powerful coding assistant, not an infallible senior engineer. Review and test everything it produces.

Misconception #10

**If it works on my machine,
it's finished**

Reality

Software runs in different environments. Differences in operating systems, dependency versions, environment variables, databases and configuration can all cause unexpected behaviour.

Professional software should behave consistently across development, testing and production.

Why it matters

Reproducibility is just as important as functionality.

Strong engineering starts with strong fundamentals.

Many bugs, security issues and debugging sessions happen because of small misunderstandings—not complex code.

The more accurately you understand the tools you use every day, the better decisions you'll make as a developer.

 ***Save this guide for future reference.***

 ***Comment "LEARNED" if you learned something new from this***